Ход выполнения лабораторной работы №13

Nullability

Теоретические вопросы

null — это значение указывающее на отсутствие ссылки. В памяти хранится как 0.

int не допускает null потому что это тип-значение, а не ссылочный тип. Число 0 для int это просто ноль, а не пустая ссылка.

string? означает что переменная может хранить либо ссылку на строку, либо null.

int? и string? работают по-разному: для string? используется нулевой адрес памяти, а для int? используется структура Nullable<int> которая содержит само число и bool флаг указывающий является ли значение null.

Оператор ! говорит компилятору "я знаю что здесь не будет null, доверяй мне". Это позволяет обойти проверки — но если значение всё-таки окажется null, программа упадёт в рантайме.

required гарантирует что поле будет инициализировано при создании объекта — компилятор не позволит создать объект без него.

Практическое задание

В файл проекта были добавлены две настройки:

<Nullable>enable</Nullable>
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>

Nullable включает строгую проверку на null — компилятор предупреждает о потенциальных null значениях. TreatWarningsAsErrors превращает все предупреждения в ошибки — программа не скомпилируется пока все проблемы не исправлены.

После включения настроек программа из предыдущей лабораторной скомпилировалась без ошибок — код был написан корректно. Поле Title в классе Book инициализировано пустой строкой "", что удовлетворяет требованиям nullability.

```
  UW PICO 5.09                    File: MySecondProject.csproj                    Modified

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>

^G Get Help    ^O WriteOut    ^R Read File   ^Y Prev Pg     ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is    ^V Next Pg     ^U UnCut Text  ^T To Spell
```

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== Все книги ===
  [свободна] Мастер и Маргарита (1967)
  [свободна] 1984 (1949)
  [свободна] Гарри Поттер (1997)
  [свободна] Преступление и наказание (1866)
  [свободна] Маленький принц (1943)

=== Берём '1984' ===
Книга взята: 1984
  [свободна] Мастер и Маргарита (1967)
  [занята] 1984 (1949)
  [свободна] Гарри Поттер (1997)
  [свободна] Преступление и наказание (1866)
  [свободна] Маленький принц (1943)

=== Берём 'Гарри Поттер' ===
Книга взята: Гарри Поттер
  [свободна] Мастер и Маргарита (1967)
  [занята] 1984 (1949)
  [занята] Гарри Поттер (1997)
  [свободна] Преступление и наказание (1866)
  [свободна] Маленький принц (1943)

=== Возвращаем '1984' ===
Книга возвращена: 1984
  [свободна] Мастер и Маргарита (1967)
  [свободна] 1984 (1949)
  [занята] Гарри Поттер (1997)
  [свободна] Преступление и наказание (1866)
  [свободна] Маленький принц (1943)

=== Возвращаем несуществующую книгу ===
Unhandled exception. System.ArgumentException: Такой книги не существует: Война и мир
   at Program.<<Main>$>g__Return|0_2(Book[] books, String title) in /Users/valentinaadvahova/Projects/univer/MySecondProject/Program.cs:line 7
1
   at Program.<Main>$(String[] args) in /Users/valentinaadvahova/Projects/univer/MySecondProject/Program.cs:line 25
valentinaadvahova@Valentinas-MacBook-Air MySecondProject %
```

Вывод

В ходе лабораторной работы было изучено понятие null и система nullability в C#. Включение <Nullable>enable</Nullable> и <TreatWarningsAsErrors>true</TreatWarningsAsErrors> помогает писать более надёжный код — компилятор сам указывает на потенциальные проблемы с null до запуска программы.